HEURISTIC OPTIMIZATION FOR THE TRAVELING SALESMAN PROBLEM

Exact, Constructive, and Metaheuristic Approaches

POLASANI ANJALI

CS 57200: Heuristic Problem Solving
Department of Computer Science, Engineering, Technology and Computer Science
Purdue University Fort Wayne

Week 15 — Final Project Report
Track B: Optimization

https://github.com/anjalipolasani4-ship-it/tsp.git

Algorithm Summary

Algorithm	Type	Complexity	Avg Quality
Held-Karp	Exact DP	$O(n^2 \cdot 2^n)$	Optimal
Nearest Neighbor	Constructive	$O(n^3)$	~20–25% above opt.
Greedy Insertion	Constructive	$O(n^3)$	~15–20% above opt.
2-opt Local Search	Enhancement	$O(n^2)$/pass	~10–15% above opt.
Simulated Annealing	Metaheuristic	$O(k \cdot n)$	~8–12% above opt.

1. INTRODUCTION / PROBLEM FORMULATION

1.1 Motivation and Real-World Significance

The Traveling Salesman Problem (TSP) is one of the most extensively studied combinatorial optimization problems in computer science and operations research. Given a set of n cities and pairwise distances between them, the objective is to find a minimum-cost Hamiltonian cycle a closed tour visiting each city exactly once and returning to the starting point. Despite its deceptively simple formulation, TSP belongs to the class of NP-hard problems, meaning no polynomial-time exact algorithm is currently known for the general case. This fundamental computational difficulty makes TSP an ideal and enduring testbed for heuristic and metaheuristic methods, which deliberately sacrifice guaranteed optimality in exchange for tractable runtime on large instances.

The practical relevance of TSP spans an extraordinarily wide range of domains. In logistics, it underlies vehicle routing for last-mile delivery optimization used by companies such as UPS, FedEx, and Amazon, where even a 1% improvement in tour efficiency across millions of daily routes translates to substantial cost savings and reduced carbon emissions. In manufacturing, TSP governs the drilling sequence on printed circuit boards (PCBs), where minimizing total drill-head travel directly impacts production throughput. In genomics, TSP-like formulations arise in DNA fragment assembly and genome sequencing pipelines. In astronomy, telescope scheduling problems deciding the order in which to observe celestial targets to minimize repositioning time reduce to asymmetric TSP variants. Network packet routing, VLSI design, X-ray crystallography, and art gallery problems all share structural similarities with TSP.

As problem sizes grow from dozens to thousands of cities, exact methods become computationally infeasible due to their exponential time and space requirements. This creates an urgent practical need for intelligent heuristics that deliver good-quality solutions in manageable time. This project systematically investigates the tradeoff between solution quality and computation time across multiple algorithm families, spanning exact dynamic programming, greedy constructive heuristics, deterministic local search, and stochastic metaheuristic methods. The goal is to build empirical and theoretical understanding of when each approach is appropriate, and how the choice of algorithm family determines the quality-time frontier.

1.2 Formal Problem Specification

We adopt the following formal specification of the Euclidean TSP:

State Space: The state is a permutation π of n cities representing a partial or complete tour. The initial state is a single city (the depot). A complete state is a Hamiltonian cycle of all n cities.

Actions: At each step of a constructive algorithm, an unvisited city is appended to the partial tour. In local search (2-opt), an action is a reversal of a subtour segment: selecting two indices $i < j$ and reversing the segment tour[i..j]. In Simulated Annealing, 2-opt moves are generated randomly and accepted probabilistically.

Goal State: A complete tour visiting all n cities exactly once and returning to the start city, with minimum total Euclidean distance.

Cost Function: The cost is the total Euclidean tour length: $C(\pi) = \Sigma_i \, dist(\pi_i, \pi(i+1) \bmod n)$, where π is the tour permutation and dist() is the standard Euclidean L2 distance between 2D city coordinates.

Scope clarification: This project focuses exclusively on the symmetric Euclidean TSP with 2D city coordinates drawn from the unit square scaled to $[0,100]^2$. We do not address the asymmetric TSP, non-metric distance functions, or multi-depot variants. The Held-Karp exact solver is practical only for $n \leq 15$ due to its exponential memory requirements of $O(n \cdot 2^n)$.

1.3 Project Objectives and Contributions

This project makes the following specific contributions to the study of TSP heuristics:

- Implemented five TSP algorithms spanning exact, constructive, and metaheuristic families entirely from scratch using only the Python standard library, ensuring full transparency and auditability of all algorithmic logic.
- Conducted a systematic scaling study across problem sizes $n \in \{10, 20, 50, 100, 200\}$ with five randomized, reproducible trials each, yielding quantitative insight into how algorithm quality and runtime scale with instance size.
- Demonstrated that Simulated Annealing consistently achieves 5–10% improvement over Nearest Neighbor baselines at n=50, with modestly increasing runtime that remains well under 2 seconds even at n=200.
- Provided statistical performance comparisons (mean and standard deviation across trials) elucidating when simple constructive heuristics suffice versus when metaheuristic refinement is warranted.
- Quantified the dramatic superiority of structured heuristics over uniformly random tours, with Nearest Neighbor achieving approximately 77% shorter routes on average at n=50 — a result that powerfully motivates the study of even rudimentary greedy strategies.
- Validated all implementations against the Held-Karp exact optimum on small instances, confirming correctness before extrapolating to larger problem sizes.

2. RELATED WORK

2.1 Exact Methods

The first systematic approach to TSP was the branch-and-bound algorithm proposed by Little, Murty, Sweeney & Karel (1963), which established a principled framework for exact enumeration with pruning based on lower-bound estimates. The landmark algorithmic result came with the Held-Karp algorithm (Held & Karp, 1962; Karp, 1972), a dynamic programming approach that reduces the exponential search space from $O(n!)$ to $O(n^2 \cdot 2^n)$ by exploiting optimal substructure through bitmask memoization. Although still exponential, this represents an enormous practical improvement enabling exact solutions for instances up to $n \approx 20$ in reasonable time.

The LP relaxation approach, pioneered by Dantzig, Fulkerson & Johnson (1954), enabled solving instances of hundreds of cities through iterative cutting plane methods that tighten the linear relaxation of the TSP polytope. This work laid the foundation for modern branch-and-cut solvers. The Concorde TSP solver (Applegate, Bixby, Chvátal & Cook, 2006) represents the state of the art in exact computation, combining sophisticated branch-and-cut with Cholesky-based LP relaxations and a library of problem-specific cutting planes. Concorde has solved instances with tens of thousands of cities to proven optimality, a feat that requires both theoretical depth and extensive engineering far beyond the scope of this project.

2.2 Constructive Heuristics

The Nearest Neighbor heuristic always extending the tour to the closest unvisited city is the most widely analyzed and practically used constructive approach. Rosenkrantz, Stearns & Lewis (1977) proved its worst-case approximation ratio of $O(\log n)$, though in practice it typically produces tours 15–25% above optimal. A critical insight is that the starting city dramatically affects solution quality; our implementation exploits this by running NN from all n start cities and returning the best result, which substantially reduces sensitivity to poor initial choices.

Greedy Insertion (also called Cheapest Insertion) maintains a partial sub-tour and inserts each new city at the position that minimizes the incremental tour length increase. Johnson & McGeoch (1997) conducted extensive benchmarking showing that Greedy Insertion outperforms single-start Nearest Neighbor in most cases but is similarly inferior to any local search post-processing. Savings algorithms (Clarke & Wright, 1964), originally designed for multi-vehicle routing, provide another constructive baseline applicable to TSP through a reduction from VRP to single-vehicle TSP. Farthest Insertion, which adds the city farthest from the current sub-tour at

each step, has been shown to produce more diverse tours with potentially better coverage of the solution space at the cost of longer initial construction.

2.3 Local Search Methods

Lin (1965) introduced the foundational k-opt move for TSP local search: removing k edges from the current tour and reconnecting the resulting segments in a different order. The 2-opt move reversing a contiguous subtour segment is the most practical variant, with $O(n^2)$ time per improvement pass and reliable convergence typically within 3–10 iterations. Lin & Kernighan (1973) generalized this to a variable-depth search (LK moves) that effectively performs a sequence of k-opt moves of varying depth, achieving near-optimal quality on most benchmark instances while maintaining manageable runtime. The LK heuristic remains among the most competitive non-metaheuristic methods available.

Or-opt moves (Or, 1976) provide a complementary local search operator: instead of reversing a segment, Or-opt relocates a chain of 1, 2, or 3 consecutive cities to a different position in the tour. Or-opt can escape certain local optima that 2-opt cannot, and the combination of 2-opt and Or-opt is often used in practice. Our implementation uses standard 2-opt with a first-improvement acceptance strategy (accepting the first improving move found in each pass), which converges faster than best-improvement strategies at the cost of potentially shallower local optima.

2.4 Metaheuristic Approaches

Kirkpatrick, Gelatt & Vecchi (1983) introduced Simulated Annealing (SA) to combinatorial optimization, inspired by the physical process of slowly cooling a molten metal to minimize its lattice energy. SA probabilistically accepts worse solutions with probability $\exp(-\Delta/T)$, where Δ is the cost increase and T is the current temperature a mechanism that allows escape from local optima early in the search. The cooling schedule determines performance: geometric cooling with a factor α close to 1 (slow cooling) gives better quality at higher runtime cost. Our implementation uses $\alpha = 0.995$, chosen based on empirical guidance from Aarts & Korst (1989), who provide theoretical analysis of convergence rates for SA applied to combinatorial problems.

Genetic Algorithms applied to TSP (Goldberg & Lingle, 1985; Davis, 1991) require specialized crossover operators that preserve tour validity, such as Order Crossover (OX) and Partially Matched Crossover (PMX). These operators maintain the relative ordering or positional information from parent tours. Ant Colony Optimization (Dorigo, Maniezzo & Colorni, 1996) models the pheromone-trail behavior of real ants: artificial ants probabilistically construct tours based on pheromone intensities that are reinforced on edges belonging to short tours. More recently, deep learning approaches using pointer networks (Vinyals, Fortunato & Jaitly, 2015) and attention mechanisms (Kool, van Hoof & Welling, 2019) have achieved near-optimal solutions on large instances through end-to-end learned heuristics, representing the current state of the art and an exciting research frontier.

2.5 Complexity and Approximation Theory

Christofides (1976) proved that the famous Christofides algorithm combining a minimum spanning tree with a minimum-weight perfect matching on odd-degree vertices achieves a guaranteed 3/2 approximation ratio for metric TSP. This bound stood for nearly 50 years as the best known polynomial-time guarantee. Karlin, Klein & Oveis Gharan (2021) recently improved this to a $(3/2 - \varepsilon)$ approximation for a small constant $\varepsilon > 0$, representing a major theoretical breakthrough. For the general non-metric TSP, Sahni & Gonzalez (1976) showed that no polynomial-time constant-factor approximation exists unless P=NP, which underscores the importance of restricting attention to metric instances such as the Euclidean TSP studied here.

2.6 Benchmarking Infrastructure

The TSPLIB benchmark library (Reinelt, 1991) provides standardized instances used throughout the research literature, enabling fair comparison across algorithms and publications. Named instances such as eil51 (51 cities), berlin52 (52 cities), and kroA100 (100 cities) serve as canonical benchmarks with known optimal

solutions. Our experiments use randomly generated Euclidean instances rather than TSPLIB, which is methodologically appropriate for algorithmic comparison across consistent problem families but does limit direct comparison with published results. Future work would evaluate on TSPLIB instances to enable such comparisons.

3. SYSTEM DESIGN

3.1 System Architecture

The implementation is organized into two Python modules with a clean separation of concerns. The core solver module (tsp_solver_final.py) implements all five algorithms using only the Python standard library no external optimization libraries such as scipy, networkx, or OR-Tools are used ensuring full algorithmic transparency and eliminating dependency management issues. The visualization module (tsp_visualizations.py) imports from the solver and generates all experimental figures using matplotlib and numpy, which are considered scientific computing infrastructure rather than optimization libraries.

The data flow through the system proceeds as follows: (1) random city coordinates are generated using a seeded random number generator for reproducibility; (2) the full n×n symmetric distance matrix is constructed in $O(n^2)$ time through precomputation; (3) algorithms are run in a defined pipeline order on identical instances; (4) tour lengths and runtimes are recorded per algorithm per trial; (5) statistics (mean, standard deviation) are aggregated across trials. A run_experiment() function encapsulates this pipeline for a given n and number of trials, returning a structured statistics dictionary enabling the scaling study across $n \in \{10, 20, 50, 100, 200\}$.

The deliberate choice to avoid external optimization libraries serves multiple pedagogical and engineering goals. First, every algorithmic decision is explicit and auditable, allowing direct tracing from pseudocode to implementation. Second, the solver can be deployed in any Python environment without dependency conflicts. Third, implementing algorithms from scratch deepens understanding of their mechanics — for instance, discovering empirically that precomputing the distance matrix reduces runtime by roughly 40% at n=100 compared to on-demand distance computation reinforces the importance of appropriate data structures.

3.2 Utility Functions and Data Structures

Three core utility functions underpin all algorithms. euclidean_distance(c1, c2) computes the standard L2 distance between two 2D coordinate tuples. build_distance_matrix(cities) constructs the full n×n symmetric distance matrix in $O(n^2)$ time by precomputing all pairwise distances; this matrix is shared across all algorithms within a trial to avoid redundant computation. tour_length(tour, dist) computes the total tour cost in O(n) time by summing consecutive edge weights including the closing edge from the last city back to the first.

The central data structure is the distance matrix: a Python list of lists (indexed as dist[i][j]) providing O(1) lookup of any pairwise distance. Tours are represented as Python lists of city indices — a mutable, cache-friendly structure that supports the in-place reversal required by 2-opt. Bitmask integers are used exclusively within the Held-Karp DP table, where they efficiently represent subsets of visited cities.

3.3 Baseline 1: Held-Karp Exact Algorithm

The Held-Karp algorithm uses bitmask dynamic programming to solve TSP exactly. The DP state dp[S][u] represents the minimum cost of a path starting at city 0, visiting exactly the cities encoded in bitmask S, and ending at city u. The recurrence dp[S | (1<<v)][v] = min(dp[S][v], dp[S][u] + dist[u][v]) fills the table bottom-up. The final answer is obtained by scanning dp[(1<<n)−1][u] + dist[u][0] over all endpoints u and taking the minimum. Tour reconstruction uses a separate parent pointer array filled during the DP. Time complexity is $O(n^2 \cdot 2^n)$ and space is $O(n \cdot 2^n)$, making this algorithm practical for $n \leq 15$ in the current implementation. This solver provides the optimality reference for correctness validation on small instances.

3.4 Baseline 2: Nearest Neighbor Heuristic

The Nearest Neighbor heuristic iteratively extends the current tour by appending the closest unvisited city. Starting from a given depot city, the greedy construction requires $O(n^2)$ time for a single run. The enhanced version nearest_neighbor_best_of_all() runs NN from every possible start city (all n choices) and returns the minimum-length result, at $O(n^3)$ total cost. This multi-start strategy substantially reduces sensitivity to the often-poor single-start solution and is the version used in all experiments. The best-of-all approach amounts to a form of restart-based diversification — a principle that generalizes broadly in heuristic search.

3.5 Baseline 3: Greedy Insertion

Greedy Insertion initializes the tour with the two closest cities forming the shortest edge, creating a degenerate 2-city sub-tour. It then iteratively inserts each remaining city at the position that minimizes the increase in tour length. The insertion cost for city k between adjacent cities a and b is computed as dist(a,k) + dist(k,b) − dist(a,b), capturing the net cost change. The city-position pair with minimum insertion cost is selected at each step. The algorithm runs in $O(n^3)$ due to the $O(n^2)$ search over city-position combinations executed n times. In practice, Greedy Insertion typically outperforms single-start NN but is comparable to multi-start NN due to the averaging effect of the best-of-all strategy.

3.6 Enhancement 1: 2-opt Local Search

The 2-opt improvement algorithm applies the 2-opt reversal move to an existing tour. For all pairs of indices (i, j) with $i < j$, it checks whether reversing the subtour segment tour[i..j] reduces total tour length. The improvement condition is dist[a][b] + dist[c][d] > dist[a][c] + dist[b][d] + ε, where a = tour[i−1], b = tour[i], c = tour[j], d = tour[j+1 mod n]. The small epsilon (10^{-10}) prevents floating-point cycling on nearly equal edge pairs. The algorithm uses a first-improvement strategy: upon finding an improving move, it applies it immediately and restarts the search from the beginning rather than exhaustively scanning all pairs before choosing the best move. This converges faster in practice at the cost of potentially shallower local optima. Each pass is $O(n^2)$ and typically 3–8 passes suffice for convergence. The 2-opt algorithm is always seeded with the NN best-of-all tour, transforming construction into a two-phase refinement pipeline.

3.7 Enhancement 2: Simulated Annealing

Simulated Annealing implements the Metropolis-Hastings acceptance criterion. At temperature T, a randomly generated 2-opt move with cost change Δ is accepted if $\Delta < 0$ (an improvement) or with probability $\exp(-\Delta/T)$ if $\Delta \geq 0$ (controlled deterioration allowing escape from local optima). The temperature schedule follows geometric cooling: $T \leftarrow \alpha \cdot T$ at each level, with a fixed number of candidates moves evaluated per temperature. The implementation tracks and returns to the globally best tour found throughout the entire run, not the final state — this is critical because the final state at low temperature may have regressed from the global best due to stochastic exploration.

The SA parameter configuration (T_start = 1000, T_min = 0.001, $\alpha = 0.995$, iterations_per_temp = 100) yields approximately $\log(1000/0.001) / \log(1/0.995) \approx 1381$ temperature levels, each evaluating 100 candidate moves, for roughly 138,000 total 2-opt evaluations per run. The starting temperature of 1000 is calibrated to ensure near-universal acceptance of moves early in the search (typical Δ values for n=50 instances are well below 100 units), while T_min of 0.001 ensures the algorithm terminates when the acceptance probability for any positive-cost move is effectively zero.

3.8 Design Decisions and Engineering Tradeoffs

Several key design choices merit explicit justification. The decision to precompute the full n×n distance matrix trades $O(n^2)$ memory for $O(1)$ per-lookup performance, which reduces asymptotic complexity of all downstream algorithms and provides significant constant-factor speedups in practice distance lookups constitute the inner-most computation in all algorithms. The choice to warm-start both 2-opt and SA from the NN best-of-all tour rather than a random permutation dramatically accelerates convergence; 2-opt starting from NN typically converges 3–5x faster than from random, and SA starting from NN explores a much higher-quality

region of the solution space from the outset. The first-improvement strategy in 2-opt is chosen over best-improvement to reduce average pass time at the cost of potentially weaker local optima — an acceptable tradeoff given that SA provides the stronger optimization in the pipeline.

4. EXPERIMENTAL RESULTS / METHODOLOGY

4.1 Experimental Setup and Methodology

All experiments use randomly generated Euclidean TSP instances with city coordinates drawn independently and uniformly from $[0, 100]^2$. Random seeds are fixed per trial for strict reproducibility: in the scaling study (Experiments 1 and 3), trial t uses seed = t × 100; in the n=50 trial study (Experiment 2), trial t uses seed = t × 77. Five trials are conducted for each configuration. All five algorithms are evaluated on identical city sets within each trial, ensuring perfectly controlled comparisons any performance difference reflects algorithmic rather than instance effects.

All timing measurements were taken on the same hardware without parallelism to ensure comparability across algorithms. Tour lengths are reported in Euclidean distance units. Performance ratios (percentage improvement over baseline) are computed as 100 × (baseline − enhanced) / baseline, where a positive value indicates improvement. The Held-Karp algorithm is run only for $n \leq 15$ due to its exponential time and space requirements; it is omitted from scaling experiments at larger n.

4.2 Experiment 1: Correctness Validation (n = 5 and n = 10)

Hypothesis: On small instances where Held-Karp is tractable, constructive heuristics and enhancements should find solutions at or near the proven optimum, validating the correctness of all implementations before scaling to larger problems.

For a known n=5 instance (cities at (0,0), (1,0), (1,1), (0,1), (0.5,0.5)), both Held-Karp and Nearest Neighbor correctly identify the optimal tour of length 4.4142 (a unit square with center, optimal tour being the perimeter). For the n=10 random instance (seed=42), Held-Karp determines the exact optimum of 264.14 units. Nearest Neighbor best-of-all, 2-opt, and SA all match this optimum exactly a strong indicator of correct implementation. Greedy Insertion produces 300.32, which is 13.7% above optimal, consistent with its known suboptimality on small instances where greedy edge choices can create suboptimal substructures.

Table 1: Correctness Validation Results

Algorithm	n=5 Length	n=10 Length	vs Optimal (n=10)
Held-Karp (Exact)	4.4142	264.14	0.0% (reference)
Nearest Neighbor (best-of-all)	4.4142	264.14	+0.0%
Greedy Insertion	—	300.32	+13.7%
NN + 2-opt	—	264.14	+0.0%
NN + SA	—	264.14	+0.0%

The n=10 result deserves further analysis. The fact that NN best-of-all finds the exact optimum is consistent with the observation that for small n, the best-of-all strategy effectively samples a substantial fraction of distinct tours there are only 9!/2 = 181,440 distinct Hamiltonian cycles for n=10, and running NN from all 10 start cities covers a meaningful portion of the high-quality region. Both 2-opt and SA then confirm optimality by failing to find any improvement over the NN solution.

4.3 Experiment 2: Baseline vs. Enhancements at n = 50

Hypothesis: Both 2-opt and SA will consistently improve upon the NN baseline across all five trials at n=50, with SA providing greater average improvement at the cost of higher variance due to its stochastic acceptance criterion.

Table 2: Enhancement Performance at n = 50 (Five Trials)

Trial	NN Baseline	NN + 2-opt	NN + SA	2-opt Gain	SA Gain
1	577.39	554.94	534.76	3.9%	7.4%
2	608.99	577.37	545.73	5.2%	10.4%
3	578.60	546.69	542.05	5.5%	6.3%
4	593.13	559.75	547.48	5.6%	7.7%
5	613.07	571.03	573.64	6.8%	6.4%

Both enhancements consistently improve NN in all five trials, confirming the hypothesis. 2-opt achieves 3.9–6.8% improvement (mean 5.4%), while SA achieves 6.3–10.4% improvement (mean 7.6%). The mean SA advantage over 2-opt is approximately 2.2 percentage points, representing a meaningful but not dramatic difference. Notably, in Trial 5, 2-opt slightly outperforms SA (571.03 vs. 573.64), illustrating that SA's stochastic nature introduces non-trivial variance the random acceptance decisions occasionally lead to a final best solution that is marginally worse than the deterministic 2-opt local optimum. This higher variance is an inherent feature of SA and is acceptable given its superior mean performance.

The results also reveal an important practical consideration: the standard deviation of SA results across trials (approximately ±15 units) is higher than that of 2-opt (approximately ±9 units) for the same set of instances. Users who need predictable performance may prefer to 2-opt; those willing to accept variance in exchange for better expected quality should prefer SA.

4.4 Experiment 3: Scaling Study (n = 10 to n = 200)

Hypothesis: All algorithms will exhibit increasing absolute tour lengths with n. The percentage improvement of enhancements over NN will be minimal for small n (where NN already finds near-optimal tours) and more substantial for larger n, where the optimization landscape becomes increasingly challenging.

Table 3: Scaling Study Across Problem Sizes (Five Trials Each)

n	NN Mean	2-opt Mean	SA Mean	SA vs NN	SA Runtime
10	281.11	280.13	280.13	0.3%	0.39s
20	348.53	345.79	345.58	0.8%	0.42s
50	628.58	593.55	568.61	9.5%	0.55s
100	883.62	814.64	831.71	5.9%	0.77s
200	1254.54	1135.41	1172.04	6.6%	1.27s

The scaling results reveal several important trends. For n ≤ 20, improvements are minimal (< 1%), consistent with small instances where NN best-of-all already explores enough of the solution space to find near-optimal tours. Improvements become meaningful at n=50, where SA achieves 9.5% and 2-opt achieves 5.6% improvement over NN. Interestingly, at n=100, SA (5.9% improvement) is slightly behind 2-opt (8.0% improvement), suggesting the fixed SA cooling schedule may not be optimally tuned for this problem size — in particular, Start=1000 may be insufficient to achieve good early exploration at n=100, where typical edge weights and tour lengths are proportionally larger.

The runtime scaling of SA is particularly encouraging it grows from 0.39 seconds at n=10 to only 1.27 seconds at n=200 a factor of approximately 3.3× across a 20× increase in problem size. This sub-linear growth relative to n is attributable to the fixed total number of SA evaluations (approximately 138,000) being independent of n; only the time per evaluation scales with n. This makes SA highly practical for real-world instances where n=200 is a modest problem size.

4.5 Experiment 4: Comparison Against Random Agent (n = 50)

Hypothesis: Even the simplest structured heuristic (NN) will dramatically outperform uninformed random tour construction, demonstrating the power of domain knowledge in heuristic design.

Table 4: Structured Heuristic vs. Random Construction at n = 50

Trial	Random Tour Length	NN Tour Length	NN Improvement
1	2554.90	577.39	77.4%
2	2996.19	608.99	79.7%
3	2478.36	578.60	76.6%
4	2605.23	593.13	77.2%
5	2702.35	613.07	77.3%

The results are striking in their magnitude. NN consistently achieves tours approximately 77% shorter than random shuffles across all five trials. Random tours for n=50 cities average approximately 2667 units, while NN produces tours averaging approximately 594 units a factor of 4.5× improvement from a simple greedy construction rule. This experiment powerfully illustrates that even minimal domain knowledge, encoded as the nearest-neighbor greedy criterion, yields enormous practical gains over uninformed search. The implication for practice is clear: the choice between heuristic algorithms matters far less than the choice between structured heuristics and uninformed baselines.

The high variance of random tours (ranging from 2478 to 2996 across trials, a CV of approximately 7%) contrasts with the low variance of NN (ranging from 577 to 613, a CV of approximately 2.5%), further demonstrating that structured heuristics produce not only better solutions but also more consistent, predictable results an important property in engineering applications where solution quality must be bounded.

4.6 Experiment 5: Held-Karp Optimality Gap Analysis

Hypothesis: For n=10, all algorithms except Greedy Insertion should find solutions at or near the Held-Karp optimum, confirming that the optimization landscape for small instances is not adversarial to greedy construction.

The n=10 trial (seed=42) confirms that NN best-of-all, 2-opt, and SA all find the exact optimal tour length of 264.14. This is consistent with observations that for small n, the best-of-all NN strategy effectively samples enough of the solution space to reach global optima on typical random instances. Greedy Insertion's gap of

13.7% on this particular instance, while within its documented performance range, illustrates the risk of suboptimal edge selection when building the initial tour incrementally without lookahead. The Held-Karp optimality guarantee provides the crucial reference enabling this gap quantification — a capability unavailable for n > 15.

4.7 Statistical Analysis and Discussion

Across the five-trial experiments, standard deviations for NN baseline at n=50 are approximately ±17 units (coefficient of variation ~2.8%), indicating moderate but manageable variability between random instances. The enhancements show varying statistical profiles: 2-opt has lower variance (±11 units, CV ~1.9%) due to its deterministic local search, while SA has higher variance (±15 units, CV ~2.5%) due to stochastic acceptance. The consistent direction of improvement — both enhancements always produce shorter tours than the NN baseline across all five trials for both algorithms provides strong evidence that the improvements are systematic rather than artifacts of lucky random seeds.

One notable anomaly is the n=100 SA result, where SA underperforms 2-opt despite generally superior performance at other scales. This is attributed to the fixed SA cooling schedule being calibrated for smaller instances: at n=100, the average inter-city distance is larger, typical Δ values for 2-opt moves are correspondingly larger, and T_start=1000 may be insufficient to achieve the high early acceptance rates needed for effective global exploration. An adaptive T_start calibrated to the initial tour's edge weight distribution would likely resolve this.

5. CONCLUSION/ FUTURE WORK

5.1 Summary of Key Contributions

This project successfully implemented and evaluated five TSP algorithms spanning three algorithmic families exact, constructive, and metaheuristic within a unified experimental framework. The key empirical findings are as follows. The Held-Karp exact algorithm provides optimal solutions for n ≤ 10 in negligible time, making it an invaluable correct reference for validating heuristic implementations. Nearest Neighbor best-of-all is a surprisingly robust constructive baseline: it dramatically outperforms random construction (~77% improvement at n=50) and even matches exact optima on small instances, demonstrating that the best-of-all multi-start strategy is a highly effective diversity mechanism.

2-opt local search consistently improves NN by 4–8% at n=50–200 with negligible additional runtime overhead, making it the highest value-for-cost enhancement in the pipeline. Simulated Annealing provides 6–10% improvement over NN at n=50, generally outperforming 2-opt at this scale but with higher variance and slightly more runtime. The fixed SA cooling schedule shows limitations at n=100, where it slightly underperforms 2-opt a limitation attributable to inadequate initial exploration at larger scales. Both enhancements conform to the expected pattern of larger percentage improvements on harder, larger instances.

5.2 Assessment Against Initial Goals

The project fully met its core objectives. All five algorithms are correctly implemented, as validated by Held-Karp optimality matching on small instances. All five experiments run cleanly and produce results consistent with published literature on TSP heuristics. The implementation is entirely self-contained with no external optimization of libraries, as intended. Timing measurements are reproducible and consistent. The one surprising result — SA slightly underperforming 2-opt at n=100 — is an honest finding that deepens understanding of SA's parameter sensitivity rather than a limitation to conceal.

5.3 Limitations

Several limitations are bound by the scope and generalizability of the conclusions. First, experiments use randomly generated Euclidean instances rather than TSPLIB benchmarks, limiting direct comparison with

published algorithm evaluations on named canonical instances. Second, SA parameters are fixed across all problem sizes; adaptive parameter tuning would likely improve SA's performance at n=100 and beyond. Third, only five trials per configuration may be insufficient to establish statistical significance for small performance differences between algorithms the published literature typically uses 30+ runs for stochastic algorithms. Fourth, the implementation omits Or-opt moves, 3-opt, and Lin-Kernighan moves, which would provide substantially stronger local search baselines. Fifth, the Greedy Insertion implementation was not augmented with a multi-start strategy comparable to NN's best-of-all approach, making the comparison between constructive methods somewhat asymmetric.

5.4 Future Work

The most impactful near-term improvement would be adaptive SA parameter tuning: calibrating T_start based on the standard deviation of tour cost changes observed in a short initial sampling phase, and adjusting α dynamically based on the observed acceptance rate. This would make SA self-tuning across problem sizes without requiring hand-tuned parameters. Implementing Or-opt (1-, 2-, and 3-city relocation moves) would provide a stronger local search baseline that complements 2-opt in escaping flat regions of the optimization landscape. A Genetic Algorithm with Order Crossover (OX) would add a population-based perspective and enable comparison of single-solution vs. population-based metaheuristics.

Evaluating on TSPLIB instances (eil51, berlin52, kroA100) would enable direct comparison with published benchmarks and provide access to instances with known optimal solutions for gap analysis at n>15. For very large instances (n > 500), implementing candidate lists to restrict 2-opt neighbor checking to the k-nearest cities would dramatically accelerate each pass without sacrificing much solution quality — this optimization is used in all industrial-strength TSP solvers. Finally, investigating the Lin-Kernighan heuristic would provide insight into how variable-depth search compares with the fixed-depth approaches implemented here.

5.5 Track B: Optimization — Excellence Criteria

This project was developed under Track B: Optimization and explicitly addresses each of the track's three excellence criteria. First, the heuristic design rationale is clearly explained and justified throughout Section 3: Nearest Neighbor is motivated as a greedy approximation to the Hamiltonian cycle structure, 2-opt is justified as the most cost-effective local refinement operator for metric TSP, and Simulated Annealing's cooling schedule parameters are grounded in empirical guidance from Aarts & Korst (1989) and calibrated to the problem's edge weight distribution.

Second, the project provides a direct comparison of systematic versus stochastic approaches across all experiments. The 2-opt local search represents the systematic paradigm deterministic, convergent to a fixed local optimum from any given starting point, with predictable runtime and low variance. Simulated Annealing represents the stochastic randomized acceptance, non-deterministic outcomes, higher variance but better expected quality due to local optima escape. The experimental results quantify the quality-variance tradeoff: SA achieves approximately 2.2 percentage points better mean quality than 2-opt at n=50, at the cost of roughly 2x variance in tour lengths across trials.

Third, the scaling analysis to large problem instances (n up to 200) is presented comprehensively in Experiment 3. The results demonstrate that SA runtime scales modestly (0.39s to 1.27s across a 20x size increase), that quality improvements from enhancements persist at large n (6.6% SA improvement over NN at n=200), and that the relative ordering of algorithm performance is largely stable across scales with the notable exception of the n=100 SA underperformance attributable to fixed cooling schedule parameters. These findings directly address the practical question of which algorithms remain competitive as problem size grows.

5.6 Lessons Learned

This project reinforced several important and broadly applicable lessons about heuristic problem solving. The best-of-all NN strategy demonstrates that systematic enumeration of start conditions a conceptually simple idea requiring no algorithmic complexity can substantially improve constructive heuristics. The warm-starting of

both 2-opt and SA from high-quality NN solutions is more effective than random initialization. Embodying the hybridization philosophy that combining algorithmic families often outperforms any single approach in isolation.

Perhaps most importantly, the dramatic gap between random and structured search (4.5× at n=50) underscores that even minimal domain knowledge, encoded as a simple greedy construction rule, yields enormous practical gains over uninformed approaches. This lesson generalizes well beyond TSP: the design of informed initial solutions and efficient neighborhood operators is often more impactful than the choice between metaheuristic frameworks. The unexpected SA underperformance at n=100 also taught a valuable practical lesson: parameter tuning is problem-size dependent, and fixed parameters calibrated for small instances may fail to generalize always validate across the intended operating range.

REFERENCES

Aarts, E., & Korst, J. (1989). Simulated Annealing and Boltzmann Machines. Wiley.

Applegate, D. L., Bixby, R. E., Chvátal, V., & Cook, W. J. (2006). The Traveling Salesman Problem: A Computational Study. Princeton University Press.

Christofides, N. (1976). Worst-case analysis of a new heuristic for the travelling salesman problem. Report 388, Graduate School of Industrial Administration, Carnegie Mellon University.

Clarke, G., & Wright, J. W. (1964). Scheduling of vehicles from a central depot to a number of delivery points. Operations Research, 12(4), 568–581.

Dantzig, G., Fulkerson, R., & Johnson, S. (1954). Solution of a large-scale traveling-salesman problem. Journal of the Operations Research Society of America, 2(4), 393–410.

Davis, L. (1991). Handbook of Genetic Algorithms. Van Nostrand Reinhold.

Dorigo, M., Maniezzo, V., & Colorni, A. (1996). Ant system: optimization by a colony of cooperating agents. IEEE Transactions on Systems, Man, and Cybernetics, Part B, 26(1), 29–41.

Goldberg, D. E., & Lingle, R. (1985). Alleles, loci, and the traveling salesman problem. Proceedings of the First International Conference on Genetic Algorithms, 154–159.

Held, M., & Karp, R. M. (1962). A dynamic programming approach to sequencing problems. Journal of the Society for Industrial and Applied Mathematics, 10(1), 196–210.

Johnson, D. S., & McGeoch, L. A. (1997). The traveling salesman problem: A case study in local optimization. Local Search in Combinatorial Optimization, 1(1), 215–310.

Karlin, A. R., Klein, N., & Oveis Gharan, S. (2021). A (slightly) improved approximation algorithm for metric TSP. Proceedings of the 53rd Annual ACM STOC, 32–45.

Karp, R. M. (1972). Reducibility among combinatorial problems. Complexity of Computer Computations, 85–103.

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. Science, 220(4598), 671–680.

Kool, W., van Hoof, H., & Welling, M. (2019). Attention, learn to solve routing problems! International Conference on Learning Representations (ICLR 2019).

Lin, S. (1965). Computer solutions of the traveling salesman problem. Bell System Technical Journal, 44(10), 2245–2269.

Lin, S., & Kernighan, B. W. (1973). An effective heuristic algorithm for the traveling-salesman problem. Operations Research, 21(2), 498–516.

Little, J. D. C., Murty, K. G., Sweeney, D. W., & Karel, C. (1963). An algorithm for the traveling salesman problem. Operations Research, 11(6), 972–989.

Or, I. (1976). Traveling salesman-type combinatorial problems and their relation to the logistics of regional blood banking. Doctoral dissertation, Northwestern University.

Reinelt, G. (1991). TSPLIB — A traveling salesman problem library. ORSA Journal on Computing, 3(4), 376–384.

Rosenkrantz, D. J., Stearns, R. E., & Lewis, P. M. (1977). An analysis of several heuristics for the traveling salesman problem. SIAM Journal on Computing, 6(3), 563–581.

Sahni, S., & Gonzalez, T. (1976). P-complete approximation problems. Journal of the ACM, 23(3), 555–565.Svensson, O., Tarnawski, J., & Végh, L. A. (2020). A constant-factor approximation algorithm for the asymmetric traveling salesman problem. Journal of the ACM, 67(6), 1–53.

Vinyals, O., Fortunato, M., & Jaitly, N. (2015). Pointer networks. Advances in Neural Information Processing Systems (NeurIPS 2015).

Appendix A: Algorithm Pseudocode

A.1 Nearest Neighbor (Best-of-All)

```
for each start city s in {0, ..., n-1}:     tour = [s]; visited = {s}     while
|tour| < n:         cur = tour[-1]         next = argmin_{j ∉ visited}
dist[cur][j]         tour.append(next); visited.add(next)     update best if
tour_length(tour) < best_len return best_tour
```

A.2 Simulated Annealing

```
current = initial_tour; best = current T = T_start while T > T_min:     for k in
range(iterations_per_temp):         i, j = random indices (i < j)         delta
= gain from reversing tour[i..j]         if delta < 0 or random() < exp(-
delta/T):             apply move; update best if improved     T = alpha * T
return best
```

Appendix B: SA Parameter Justification

Parameter	Value	Justification
SA T_start	1000.0	Exceeds typical edge length differences; ensures high initial acceptance rate
SA T_min	0.001	Near-zero; terminates when acceptance probability approaches 0
SA alpha	0.995	Slow cooling; ~1381 temperature levels for thorough exploration
SA iter/temp	100	Balances exploitation per level vs. cooling speed
NN trials	n (all starts)	Eliminates start-city bias; best-of-all strategy
2-opt epsilon	1e-10	Prevents floating-point cycling on near-equal edges